

EmbedUR Networks Training

Assessment – 2

Module – 3: Layer 2 Protocols and Ethernet

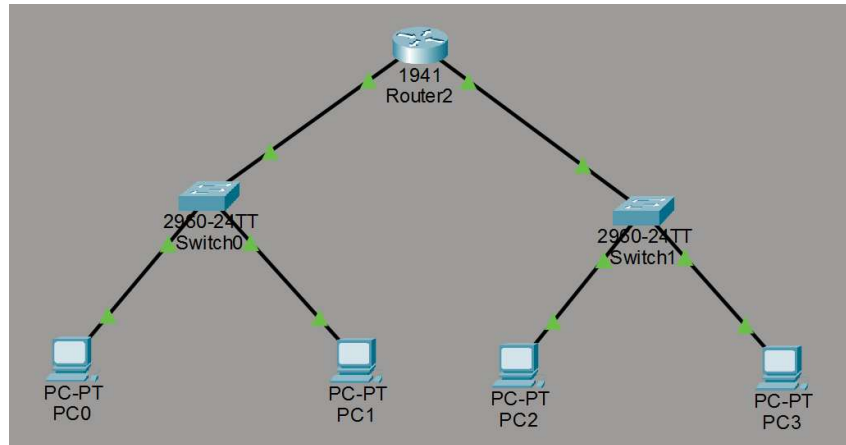
&

Module – 4 : Switching Techniques

Name: Kishore S

1. Simulate a small network with switches and multiple devices. Use ping to generate traffic and observe the MAC address table of the switch. Capture packets using Wireshark to analyse Ethernet frames and MAC addressing.

Simple Network with Cisco Packet Tracer:



Router Configuration:

```
Router2
Physical Config CLI Attributes

249856K bytes of ATA System CompactFlash 0 (Read/Write)

--- System Configuration Dialog ---
Would you like to enter the initial configuration dialog? [yes/no]: no

Press RETURN to get started!

Router>enable
Router#
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#
Router(config)#interface g0/0
Router(config-if)#
Router(config-if)#ip address 192.168.10.1 255.255.255.0
Router(config-if)#
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

Router(config-if)#
Router(config-if)#exit
Router(config)#
Router(config)#interface g0/1
Router(config-if)#ip address 192.168.20.1 255.255.255.0
Router(config-if)#
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up

Router(config-if)#
```

PC Configurations:

PC0:

PC0

Physical

Config

Desktop

Programming

Attributes

IP Configuration

X

InterfaceFastEthernet0

IP Configuration

DHCP

Static

IPv4 Address192.168.10.10

Subnet Mask255.255.255.0

Default Gateway192.168.10.1

DNS Server0.0.0.0

IPv6 Configuration

Automatic

Static

IPv6 Address

Link Local AddressFE80::260:70FF:FE6D:E159

Default Gateway

DNS Server

802.1X

Use 802.1X Security

AuthenticationMD5

Username

Password

PC1:

PC1

Physical

Config

Desktop

Programming

Attributes

IP Configuration

X

InterfaceFastEthernet0

IP Configuration

DHCP

Static

IPv4 Address192.168.10.11

Subnet Mask255.255.255.0

Default Gateway192.168.10.1

DNS Server0.0.0.0

IPv6 Configuration

Automatic

Static

IPv6 Address

Link Local AddressFE80::202:16FF:FEA5:833D

Default Gateway

DNS Server

802.1X

Use 802.1X Security

AuthenticationMD5

Username

Password

PC2:

PC2

PhysicalConfigDesktopProgrammingAttributes

IP Configuration

InterfaceFastEthernet0

IP Configuration

DHCP

Static

IPv4 Address192.168.20.10

Subnet Mask255.255.255.0

Default Gateway192.168.20.1

DNS Server0.0.0.0

IPv6 Configuration

Automatic

Static

IPv6 Address

Link Local AddressFE80::260:47FF:FE82:8948

Default Gateway

DNS Server

802.1X

Use 802.1X Security

AuthenticationMD5

Username

Password

PC3:

PC3

PhysicalConfigDesktopProgrammingAttributes

IP Configuration

InterfaceFastEthernet0

IP Configuration

DHCP

Static

IPv4 Address192.168.20.11

Subnet Mask255.255.255.0

Default Gateway192.168.20.1

DNS Server0.0.0.0

IPv6 Configuration

Automatic

Static

IPv6 Address

Link Local AddressFE80::260:70FF:FE8D:1CB

Default Gateway

DNS Server

802.1X

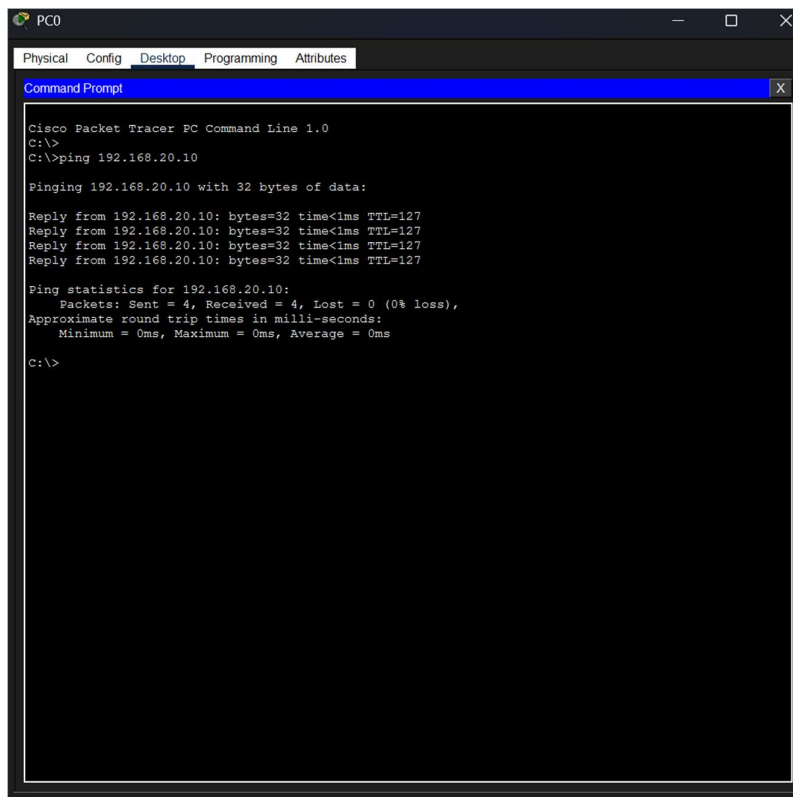
Use 802.1X Security

AuthenticationMD5

Username

Password

Simple Ping from PC0 to PC2:



```
PC0
Physical Config Desktop Programming Attributes
Command Prompt
Cisco Packet Tracer PC Command Line 1.0
C:\>
C:\>ping 192.168.20.10

Pinging 192.168.20.10 with 32 bytes of data:

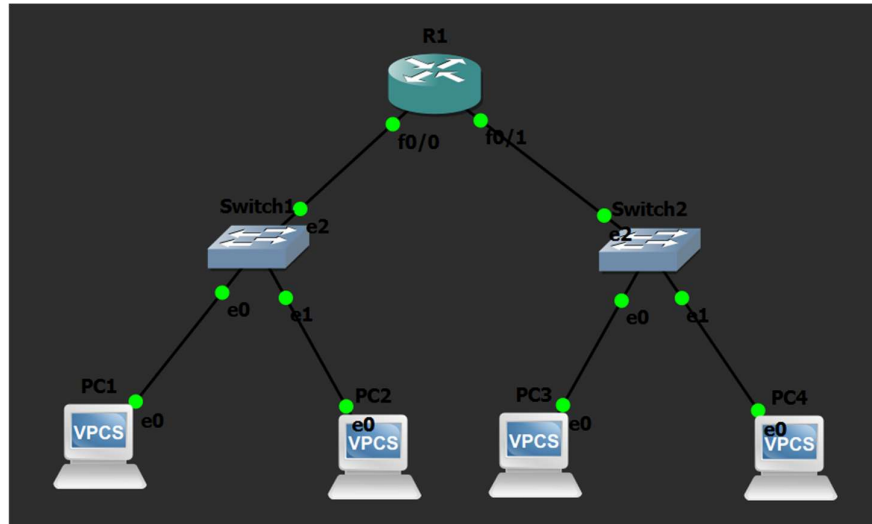
Reply from 192.168.20.10: bytes=32 time<1ms TTL=127
Reply from 192.168.20.10: bytes=32 time<1ms TTL=127
Reply from 192.168.20.10: bytes=32 time<1ms TTL=127
Reply from 192.168.20.10: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.20.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>
```

2. Capture and analyze Ethernet frames using Wireshark. Inspect the structure of the frame, including destination and source MAC addresses, Ethertype, payload, and FCS. Use GNS3 or Packet Tracer to simulate network traffic.

Topology:



Router Configuration:

```
R1
R1#sh ip int brief
Interface              IP-Address      OK? Method Status    Prot
FastEthernet0/0        unassigned      YES unset  administratively down down
FastEthernet0/1        unassigned      YES unset  administratively down down

R1#en
R1#config
Configuring from terminal, memory, or network [terminal]?
Enter configuration commands, one per line. End with CNTL/Z.
R1(config)#
R1(config)#int f0/0
R1(config-if)#
R1(config-if)#ip address 192.168.1.0 255.255.255.0
Bad_mask /24 for address 192.168.1.0
R1(config-if)#ip address 192.168.0.1 255.255.255.0
R1(config-if)#
R1(config-if)#no shut
R1(config-if)#
R1(config-if)#
*Mar 11 12:20:59.075: %LINK-3-UPDOWN: Interface FastEthernet0/0, changed state to up
*Mar 11 12:21:00.075: %LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to up
R1(config-if)#exit
R1(config)#exit
```

Configure PCs:

```
Kishore_PC - PuTTY
Copyright (c) 2007-2014, Paul Meng (mirnshi@gmail.com)
All rights reserved.

vPCS is free software, distributed under the terms of the "BSD" licence.
Source code and license can be found at vpcs.sf.net.
For more information, please visit wiki.freecode.com.cn.

Press '?' to get help.

Executing the startup file

PC2> ip 192.168.10.11/24 192.168.10.1
Checking for duplicate address...
PC1 : 192.168.10.11 255.255.255.0 gateway 192.168.10.1

PC2> show ip

NAME      : PC2[1]
IP/MASK    : 192.168.10.11/24
GATEWAY    : 192.168.10.1
DNS        :
MAC        : 00:50:79:66:68:01
LPORT     : 10010
RHOST:PORT : 127.0.0.1:10011
MTU        : 1500

PC2>
```

Ping Test:

```
Kishore_PC - PuTTY

PC2>
PC2> ping 192.168.20.10
192.168.20.10 icmp_seq=1 timeout
192.168.20.10 icmp_seq=2 timeout
84 bytes from 192.168.20.10 icmp_seq=3 ttl=63 time=33.795 ms
84 bytes from 192.168.20.10 icmp_seq=4 ttl=63 time=33.528 ms
84 bytes from 192.168.20.10 icmp_seq=5 ttl=63 time=33.370 ms

PC2> ping 192.168.20.10
84 bytes from 192.168.20.10 icmp_seq=1 ttl=63 time=33.554 ms
84 bytes from 192.168.20.10 icmp_seq=2 ttl=63 time=33.013 ms
84 bytes from 192.168.20.10 icmp_seq=3 ttl=63 time=31.220 ms
84 bytes from 192.168.20.10 icmp_seq=4 ttl=63 time=33.968 ms
84 bytes from 192.168.20.10 icmp_seq=5 ttl=63 time=32.330 ms

PC2>
```

Frames Captured from Wireshark:

*Standard input [PC1 Ethernet0 to Switch1 Ethernet0]						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	ca:01:4d:94:00:08	CDP/VTP/DTP/PagP/UD...	CDP	377	Device ID: R1 Port ID: FastEthernet0/0
2	17.834021	Private_66:68:00	Broadcast	ARP	64	Who has 192.168.10.11? (ARP Probe)
3	17.836926	Private_66:68:01	Private_66:68:00	ARP	64	192.168.10.11 is at 00:50:79:66:68:01
4	17.848029	0.0.0.0	192.168.10.11	ICMP	98	Echo (ping) request id=0x02fd, seq=1/256, ttl=64 (reply in 9)
5	17.851065	Private_66:68:01	Broadcast	ARP	64	Who has 192.168.10.1? Tell 192.168.10.11
6	18.858835	Private_66:68:01	Broadcast	ARP	64	Who has 192.168.10.1? Tell 192.168.10.11
7	19.873105	Private_66:68:01	Broadcast	ARP	64	Who has 192.168.10.1? Tell 192.168.10.11
8	19.874149	0.0.0.0	192.168.10.11	ICMP	98	Echo (ping) request id=0x04fd, seq=2/512, ttl=64 (no response found!)
9	20.887750	192.168.10.11	0.0.0.0	ICMP	98	Echo (ping) reply id=0x02fd, seq=1/256, ttl=64 (request in 4)
10	21.899563	0.0.0.0	192.168.10.11	ICMP	98	Echo (ping) request id=0x06fd, seq=3/768, ttl=64 (no response found!)
11	23.926541	0.0.0.0	192.168.10.11	ICMP	98	Echo (ping) request id=0x08fd, seq=4/1024, ttl=64 (no response found!)
12	25.950245	0.0.0.0	192.168.10.11	ICMP	98	Echo (ping) request id=0x0afd, seq=5/1280, ttl=64 (no response found!)
13	59.427865	ca:01:4d:94:00:08	CDP/VTP/DTP/PagP/UD...	CDP	377	Device ID: R1 Port ID: FastEthernet0/0
14	114.273068	ca:01:4d:94:00:08	CDP/VTP/DTP/PagP/UD...	CDP	377	Device ID: R1 Port ID: FastEthernet0/0

```

▼ Frame 3: Packet, 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on inter
  Section number: 1
  ▶ Interface id: 0 (-)
    Encapsulation type: Ethernet (1)
    Arrival Time: Mar 11, 2026 11:00:57.192720000 India Standard Time
    UTC Arrival Time: Mar 11, 2026 05:30:57.192720000 UTC
    Epoch Arrival Time: 1773207057.192720000
    [Time shift for this packet: 0.000000000 seconds]
    [Time delta from previous captured frame: 15.568000 milliseconds]
    [Time delta from previous displayed frame: 15.568000 milliseconds]
    [Time since reference or first frame: 8.494022000 seconds]
    Frame Number: 3
    Frame Length: 60 bytes (480 bits)
    Capture Length: 60 bytes (480 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:arp]
    Character encoding: ASCII (0)
    [Coloring Rule Name: ARP]

```

Sender and Receiver Info:

```


- Ethernet II, Src: Private_66:68:01 (00:50:79:66:68:01), Dst: Broadcast (ff:ff:ff:ff:  
    Destination: Broadcast (ff:ff:ff:ff:ff:ff)  
    Source: Private_66:68:01 (00:50:79:66:68:01)  
        Type: ARP (0x0806)  
        [Stream index: 3]  
        Padding: 00000000000000000000000000000000  
        Frame check sequence: 0x00000000 [unverified]  
        [FCS Status: Unverified]
- Address Resolution Protocol (request)  
    Hardware type: Ethernet (1)  
    Protocol type: IPv4 (0x0800)  
    Hardware size: 6  
    Protocol size: 4  
    Opcode: request (1)  
    Sender MAC address: Private_66:68:01 (00:50:79:66:68:01)  
    Sender IP address: 192.168.10.11  
    Target MAC address: Broadcast (ff:ff:ff:ff:ff:ff)  
    Target IP address: 192.168.10.1

```


3. Configure static IP addresses, modify MAC addresses, and verify network connectivity using ping and ifconfig commands.

Step - 1: Create a Simple Network with 3 interfaces and a Switch with Mininet:

```
kishore@kishore-virtual-machine: ~  
kishore@kishore-virtual-machine:~$ sudo mn --topo single,3  
*** No default OpenFlow controller found for default switch!  
*** Falling back to OVS Bridge  
*** Creating network  
*** Adding controller  
*** Adding hosts:  
h1 h2 h3  
*** Adding switches:  
s1  
*** Adding links:  
(h1, s1) (h2, s1) (h3, s1)  
*** Configuring hosts  
h1 h2 h3  
*** Starting controller  
  
*** Starting 1 switches  
s1 ...  
*** Starting CLI:  
mininet>
```

Step - 2: Configure Static IPs to the Interfaces H1 and H2:

```
kishore@kishore-virtual-machine: ~  
mininet> h1 ifconfig h1-eth0 10.0.0.1 netmask 255.255.255.0  
mininet>  
mininet>  
mininet> h2 ifconfig h2-eth0 10.0.0.2 netmask 255.255.255.0  
mininet>  
mininet> h1 ifconfig  
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 10.0.0.1 netmask 255.255.255.0 broadcast 10.0.0.255  
    inet6 fe80::182f:c8ff:fec1:efa8 prefixlen 64 scopeid 0x20<link>  
    ether 1a:2f:c8:c1:ef:a8 txqueuelen 1000 (Ethernet)  
    RX packets 44 bytes 4694 (4.6 KB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 11 bytes 866 (866.0 B)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 0 bytes 0 (0.0 B)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 0 bytes 0 (0.0 B)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
mininet> h2 ifconfig  
h2-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 10.0.0.2 netmask 255.255.255.0 broadcast 10.0.0.255  
    inet6 fe80::fc63:e2ff:fe62:a40b prefixlen 64 scopeid 0x20<link>  
    ether fe:63:e2:62:a4:0b txqueuelen 1000 (Ethernet)  
    RX packets 44 bytes 4694 (4.6 KB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 11 bytes 866 (866.0 B)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 0 bytes 0 (0.0 B)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 0 bytes 0 (0.0 B)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Step - 3: Modify the MAC Addresses:

Initial MAC of H1:

```
mininet> h1 ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.1 netmask 255.255.255.0 broadcast 10.0.0.255
    inet6 fe80::182f:c8ff:fec1:efa8 prefixlen 64 scopeid 0x20<link>
    ether 1a:2f:c8:c1:ef:a8 txqueuelen 1000 (Ethernet)
    RX packets 44 bytes 4694 (4.6 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 11 bytes 866 (866.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Bringing the Interface Down:

```
mininet>
mininet> h1 ifconfig h1-eth0 down
mininet>
```

Edit the MAC Address and bring the Interface Back up Again:

```
mininet> h1 ifconfig h1-eth0 hw ether 00:00:00:00:00:11
mininet>
mininet> h1 ifconfig h1-eth0 up
mininet>
```

H1 with New MAC Address:

```
mininet>
mininet> h1 ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.1 netmask 255.255.255.0 broadcast 10.0.0.255
    inet6 fe80::200:ff:fe00:11 prefixlen 64 scopeid 0x20<link>
    ether 00:00:00:00:00:11 txqueuelen 1000 (Ethernet)
    RX packets 46 bytes 4914 (4.9 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 18 bytes 1492 (1.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

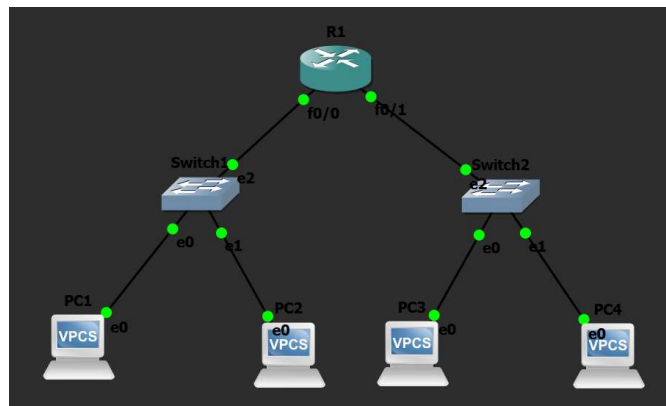
Ping Test:

```
mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.24 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.166 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.098 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.118 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.088 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.066 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.133 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.157 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.149 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.130 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.177 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=0.085 ms
^C
--- 10.0.0.2 ping statistics ---
12 packets transmitted, 12 received, 0% packet loss, time 11224ms
rtt min/avg/max/mdev = 0.066/0.217/1.244/0.311 ms
mininet>
```

4. Troubleshoot Ethernet Communication with ping and traceroute -> Using cisco packet tracer:

- . Create a simple LAN setup with two Linux machines connected via a switch.
- . Ping from one machine to the other. If it fails, use ifconfig to ensure the IP addresses are configured correctly.
- . Use traceroute to identify where the packets are being dropped if the ping fails.

Network Topology:



Ping PC1 to PC4:

```
PC1>
PC1> ping 192.168.20.10
host (192.168.20.10) not reachable
```

Ping PC1 to PC2

```
PC1> ping 192.168.10.11
192.168.10.11 icmp_seq=1 timeout
192.168.10.11 icmp_seq=2 timeout
192.168.10.11 icmp_seq=3 timeout
192.168.10.11 icmp_seq=4 timeout
192.168.10.11 icmp_seq=5 timeout

PC1> █
```

Ping is Getting Failed. So, checking if IP addresses are configured properly:

```
PC1> show ip
NAME       : PC1[1]
IP/MASK    : 0.0.0.0/0
GATEWAY    : 0.0.0.0
DNS        :
MAC        : 00:50:79:66:68:00
LPORT      : 10012
RHOST:PORT : 127.0.0.1:10013
MTU        : 1500
```

Hence, the IP address is not configured properly as the left subnet is configured with the IP address 192.168.10.1. So, the IP address of the device should be 192.168.10.x. So, reconfiguring the IP address:

```
PC1> ip 192.168.10.10/24 192.168.10.1
Checking for duplicate address...
PC1 : 192.168.10.10 255.255.255.0 gateway 192.168.10.1
```

Ping Test from PC1 to PC2:

```
PC1>
PC1> ping 192.168.10.11
84 bytes from 192.168.10.11 icmp_seq=1 ttl=64 time=4.308 ms
84 bytes from 192.168.10.11 icmp_seq=2 ttl=64 time=5.599 ms
84 bytes from 192.168.10.11 icmp_seq=3 ttl=64 time=5.678 ms
84 bytes from 192.168.10.11 icmp_seq=4 ttl=64 time=4.438 ms
84 bytes from 192.168.10.11 icmp_seq=5 ttl=64 time=4.746 ms
```

Ping Test from PC1 to PC3:

```
PC1> ping 192.168.20.10
84 bytes from 192.168.20.10 icmp_seq=1 ttl=63 time=33.574 ms
84 bytes from 192.168.20.10 icmp_seq=2 ttl=63 time=34.859 ms
84 bytes from 192.168.20.10 icmp_seq=3 ttl=63 time=34.744 ms
84 bytes from 192.168.20.10 icmp_seq=4 ttl=63 time=33.367 ms
84 bytes from 192.168.20.10 icmp_seq=5 ttl=63 time=33.942 ms
```

Now, everything works fine.

5. Research the Linux kernel's handling of Ethernet devices and network interfaces. Write a short report on how the Linux kernel supports Ethernet communication (referencing kernel.org documentation).

The Linux kernel provides a comprehensive networking subsystem that manages Ethernet devices and network interfaces. This subsystem enables communication between hardware network interface cards (NICs), kernel networking protocols, and user applications.

Linux Networking Subsystem

The Linux kernel networking architecture is implemented mainly in the **net/** and **drivers/net/** directories of the kernel source tree. The **net/** directory contains core networking protocols

(such as TCP/IP), while `drivers/net/` contains hardware-specific network drivers. Development and maintenance of these components are coordinated through the **netdev subsystem**.

Network Device Representation

In the kernel, each network interface is represented using the **struct net_device** data structure. This structure stores information about the device such as:

- Interface name (e.g., `eth0`)
- MAC address
- MTU (Maximum Transmission Unit)
- Device state and flags
- Pointers to driver operations

The kernel allocates and manages these structures using functions like `alloc_netdev()` and releases them with `free_netdev()` when the device is no longer in use.

Ethernet Device Drivers

Ethernet communication in Linux relies on **device drivers** that interact directly with network hardware. These drivers:

- Initialize the NIC
- Handle packet transmission and reception
- Manage interrupts and DMA buffers
- Provide statistics and configuration interfaces

Many drivers exist for various hardware manufacturers such as Intel, Mellanox, and others, all integrated within the kernel networking framework.

Packet Processing

When an Ethernet frame arrives:

1. The NIC receives the frame from the physical medium.
2. The device driver passes the packet to the kernel networking stack.
3. The kernel processes the packet using data structures such as **sk_buff**, which represents network packets inside the kernel.
4. The packet is then delivered to higher-level protocols like IP, TCP, or UDP.

Similarly, when transmitting data, packets are created by higher layers, passed through the network stack, and delivered to the NIC driver for transmission.

Communication with User Space

The Linux kernel provides mechanisms such as **Netlink sockets** to allow communication between kernel networking components and user-space tools. These tools (for example `ip` from the `iproute2` suite) allow administrators to configure network interfaces and routing tables.

6. Describe how you would configure a basic LAN interface using the `ip` command in Linux (kernel.org).

Step 1: View Available Network Interfaces

First, list all network interfaces to identify the LAN interface.

➔ `ip link show`

This command displays interfaces such as `eth0`, `enp0s3`, or `wlan0`.

Step 2: Enable the Interface

Before configuring it, bring the interface up.

➔ `sudo ip link set dev eth0 up`

This activates the network interface so it can transmit and receive packets.

Step 3: Assign an IP Address

To configure a static IP address for the LAN interface:

➔ `sudo ip addr add 192.168.1.10/24 dev eth0`

This assigns the IP address **192.168.1.10** with subnet mask **255.255.255.0** to the interface.

Step 4: Configure the Default Gateway

Add a default gateway so the system can communicate with other networks.

➔ `sudo ip route add default via 192.168.1.1 dev eth0`

This sets the router at **192.168.1.1** as the gateway for outgoing traffic.

Step 5: Verify Configuration

Check the configuration using:

➔ `ip addr show eth0`

verify connectivity with:

➔ `ping 192.168.1.1`

7. Use Linux to view the MAC address table of a switch (if using a Linux-based network switch). Use the bridge or ip link commands to inspect the MAC table and demonstrate a basic switch's operation.

```
kishore@kishore-virtual-machine:~$ cat /proc/net/arp
IP address      HW type    Flags     HW address    Mask     Device
192.168.31.1    0x1        0x2       d4:4d:9f:b7:35:1c  *       ens33
kishore@kishore-virtual-machine:~$
```

```
kishore@kishore-virtual-machine:~$ arp -a
jiofiber.local.html (192.168.31.1) at d4:4d:9f:b7:35:1c [ether] on ens33
kishore@kishore-virtual-machine:~$
```